

Representing the SKI Combinator Calculus in the Nock Instruction Set Architecture

N. E. Davis `~lagrev-nocfep`
ZeroAttest, Groundwire Foundation

Abstract

It has been claimed that the Nock ISA closely corresponds to the `ski` combinator calculus dressed up for nouns. In particular, opcodes 2, 1, and 0 have been said to correspond to the combinators `S`, `K`, and `I`. This claim is usually stated as an analogy, and the details are elided. We here make it into an algorithm. We give a four-rule compiler from arbitrary `ski` terms to Nock formulas using only opcodes 0, 1, and 2, prove it correct under a fixed application convention, and exercise it on the usual worked examples: the identity laws, Church booleans, the `B`, `C`, and `W` combinators, and Church numerals performing real arithmetic. The compiled output is verified against an independent Nock 4K interpreter. The construction is a constructive proof that the Nock fragment of 0, 1, 2 opcodes is Turing-complete. We are explicit about the three properties it does *not* have—laziness, normal-form readback, and sharing—each marking a boundary where the fragment is genuinely insufficient and additional tools are required.

Contents

1 Introduction

2

Manuscript submitted for review.

Address author correspondence to `~lagrev-nocfep`.

2	Preliminaries	4
3	Combinator Formulas	4
3.1	I identity	5
3.2	K constant	5
3.3	S substitution	5
3.4	B composition	6
3.5	C transposition	6
3.6	W duplicate	6
3.7	Commentary	7
4	Compilation of the Homomorphism	7
5	Worked Examples	9
5.1	Identity laws	9
5.2	Church booleans	11
5.3	The B, C, W combinators	12
5.4	Church numerals and arithmetic	13
6	Conclusion	16

1 Introduction

The Nock instruction set architecture is a combinator calculus sufficient to represent all computable functions. It has been a longstanding temptation to draw a direct analogy from the Nock ISA to the SKI combinator calculus. This analogy claims that the Nock opcodes 2, 1, and 0 correspond to the combinators S, K, and I respectively. The analogy is usually stated informally, e.g. “[there are] some subtle differences to Nock’s expression of S as opcode 2 that we will elide as being fundamentally similar, but perhaps worthy of its own monograph” ([lagrev2025](#), [lagrev2025](#)). We here deliver upon that promise and convert an Urbit legend into a concrete demonstration.

Combinator calculi build all computation from a handful of variable-free operators; the SKI system does it with three ([Curry1958](#), [Curry1958](#)), which is neither unique nor mini-

mal but balances convenience with economy.¹ Put briefly, the S combinator (“substitute”) takes three arguments and applies the first to the third, then applies the second to the third, and finally applies the result of the first application to the result of the second. The K combinator (“k’onstant”) takes two arguments and returns the first, discarding the second. The I combinator (“identity”) takes one argument and returns it unchanged. The SKI system is Turing-complete, meaning that any computable function can be expressed as a combination of these three operators.

$$\begin{aligned} S \ x \ y \ z &= x \ z \ (y \ z) \\ K \ x \ y &= x \\ I \ x &= x \end{aligned}$$

The Nock ISA is also Turing-complete, and it is natural to ask whether the SKI combinators can be represented in Nock. The answer is yes, but some details are subtle. The analogy between Nock opcodes and SKI combinators is not a simple one-to-one compilation procedure, but a homomorphism that preserves the structure of the computation.

A straightforward reading of Nock opcode 0 yielding I is uncontroversial, as [0 1] trivially returns identity. The reading of opcode 1 yielding K is also straightforward, as [1 x] returns a constant function that discards its argument (subject). The reading of opcode 2 yielding S is more subtle, as it requires a fixed convention for how the compiled combinator receives its arguments. The “subtle differences” of **lagrev2025**’s elision matter more than they appear. The opcode-to-combinator analogy is not a compilation procedure. SKI terms are curried and higher-order: combinators are routinely partially applied, and a combinator may be passed as an argument to another combinator. A symbol-for-symbol substitution—writing 2 for S, 1 for K, and 0 for I, with appropriate transposition of arguments into subject and formula—produces nothing runnable the moment S has fewer than three arguments, which is almost always. What is needed is a *homomorphism*: a translation under which Nock application of compiled terms mirrors SKI re-

¹See particularly **Smullyan1985** (**Smullyan1985**) for more on combinatory logic.

duction, with a fixed convention for how a compiled combinator receives its arguments.

We supply such a homomorphism as four rules with three small Nock formulas yielding the SKI combinators. The remainder of this article consists of worked examples, mechanical validation, a cost analysis, and an accounting of the limitations.

2 Preliminaries

We assume Nock 4K. Evaluation is the function `*[subject formula]`. We require only three opcodes from the Nock ISA:

```

:: slot: fetch the noun at tree address b
*[a 0 b]    -> /[b a]

:: quote: return b unchanged
5 *[a 1 b]    -> b

:: eval: compute a subject and a formula, then run
*[a 2 b c]   -> *[*[a b] *[a c]]

```

plus the structural distribution (autocons) rule, which fires whenever a formula's head is itself a cell:

```

*[a [b c] d] -> [*[a b c] *[a d]]

```

Slot address 1 denotes the whole subject, so `*[a 0 1] = a`.

A *value* is a Nock formula that consumes its argument as the *subject*. To apply a value v to an argument x , we evaluate

```

apply(v, x) == *[*[x v]

```

This operative convention inverts the lambda-calculus argument order, because in Nock the function is the formula and the argument is the subject.

3 Combinator Formulas

Under the convention of Section 2, each SKI combinator is a closed Nock formula. We give each, then verify its defining law by reduction.

3.1 I identity

$I\ x = x$ requires $\star[x\ I] = x$, which is slot 1:

$I = [0\ 1]$

3.2 K constant

Applying K to x must yield a value that ignores its next argument and returns its value. Thus $K\ x\ y = x$ requires $\star[x\ K] = [1\ x]$, which is built from subject x by autocons:

$K = [[1\ 1]\ [0\ 1]]$

For example,

$\star[x\ K] = \star[x\ [1\ 1]]\ \star[x\ [0\ 1]] = [1\ x]$

Then for any y , $\star[y\ [1\ x]] = x$, so $K\ x\ y = x$ holds.

3.3 S substitution

Substitution states $S\ x\ y\ z = (x\ z)(y\ z)$. The combinator accumulates x , then y , then fires on z . We construct it bottom-up.

When the fully-applied $S\ x\ y$ finally receives z as subject, it must produce $(x\ z)(y\ z)$. Under the convention, $x\ z = \star[z\ x]$ and $y\ z = \star[z\ y]$, and the outer application is $\star[\star[z\ y]\ \star[z\ x]]$. That is precisely opcode 2 with subject z , formula-b equal to y , formula-c equal to x :

$S\ x\ y = [2\ y\ x]$

$\therefore \star[z\ [2\ y\ x]] = \star[\star[z\ y]\ \star[z\ x]] = (x\ z)(y\ z)$

Working back one step, applying $S\ x$ to y (subject y , constant x) must build the noun $[2\ y\ x]$:

$S\ x = [[1\ 2]\ [0\ 1]\ [1\ x]]$

We check that $\star[y\ S\ x] = [2\ y\ x]$, since $\star[y\ [1\ 2]] = 2$, $\star[y\ [0\ 1]] = y$, and $\star[y\ [1\ x]] = x$. Working back the final step, applying S to x (subject x) must build $S\ x$; the first two parts are constants and the third, $[1\ x]$, is built from x by the same autocons trick used for K :

$$S = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

We check the evaluation: $\star[x \ S] = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & x \end{bmatrix}$
 $= S \ x.$

3.4 B composition

Any other combinator can be reached by elaborating its lambda definition to SKI and compiling, but the result is large. A named combinator is better derived directly, by the same method as the previous subsection: it becomes one builder layer per argument. The innermost layer is the opcode 2 application skeleton that fires on the final argument; each enclosing layer is an autocons that captures one argument and embeds it. We here derive B, C, and W; a similar methodology could be used for much of Smullyan's aviary (**Smullyan1994, Smullyan1994**).

For $B \ f \ g \ x = f \ (g \ x)$, the firing step computes f applied to $g \ x$, i.e. $\star[\star[x \ g] \ f]$, which is opcode 2 with g in the formula-b slot and f quoted in the formula-c slot; capturing g then f gives:

```

::  $\star[x \ B \ f \ g] = \star[\star[x \ g] \ f] = f \ (g \ x)$ 
B f g = [2 g [1 f]]
::  $\star[g \ B \ f] = [2 g [1 f]]$ 
B f = [[1 2] [0 1] [1 [1 f]]]
5 ::
B = [[1 [1 2]] [1 [0 1]] [[1 1] [[1 1] [0 1]]]]

```

3.5 C transposition

The same procedure yields C (flip):

```

::  $f \ x \ y \rightarrow f \ y \ x$ 
C = [[1 1 2] [1 [1 1] 0 1] [1 1] 0 1]

```

3.6 W duplicate

Likewise, W (duplicate):

```

::  $f \ x \rightarrow f \ x \ x$ 
W = [[1 2] [1 0 1] 0 1]

```

3.7 Commentary

The one subtlety is how a captured argument is embedded, which is dictated by how the combinator uses that argument. If the argument is applied to a computed value (as B's f is applied to $g\ x$), it occupies a formula slot and is embedded quoted, $[1\ f]$. If it is applied to the current subject (as W's f is applied to x) it must pass through as the live formula, embedded by slot, $[0\ 1]$. Quote where the argument is data; slot where the argument is the active function.

Notably, the value forms of B, C, and W use only Nock opcodes 0 and 1. The opcode 2 appears only in the formula they assemble at application time ($[2\ g\ [1\ f]]$), carried as quoted data. The combinators are pure construction; the application machinery is the structure they emit. Among the basis only S carries a 2 in its own body, because it performs two applications at once.

The size payoff over bracket abstraction is large (cell counts; see Section ?? for method):

	direct	via SKI	ratio
B	11	208	19×
C	11	208	19×
W	6	130	22×

The two forms are extensionally identical on every input tested. For any combinator known by name, direct derivation is preferred and the abstraction tax is avoided entirely; bracket abstraction remains the general fallback for arbitrary lambda terms.

4 Compilation of the Homomorphism

Let $[[t]]$ denote a *closed* Nock formula whose product is the value of the SKI term t . The compiler consists of four rules:

```

[[S]]  = [1 S]
[[K]]  = [1 K]
[[I]]  = [1 I]
[[A B]] = [2 [[B]] [[A]]]

```

with S, K, I the formulas of Section 3. The combinators are *quoted* so that $[[\cdot]]$ always denotes a value-producing computation; application is opcode 2 applied to the two compiled operands. $[[t]]$ produces the value of t ; it is closed, so the naïve approach of $*[a \ [[t]]]$ yields $\text{val}(t)$ for any subject a and is not yet an application. To apply t to Nock-noun arguments $a[0] \dots a[n]$, fold them onto the value, each quoted, by opcode 2:

```
run(t; a[1] ... a[n]) = *[0 F[n]]
```

where $F[0] = [[t]]$, $F[i] = [2 \ [1 \ a[i]] \ F[i-1]]$. Each layer $*[\cdot \ [2 \ [1 \ a[i]] \ F]]$ reduces to $*[a[i] \ \text{val}]$, the value applied to $a[i]$ as subject. Throughout the remainder of this paper, we will write $t \ a[1] \dots a[n] = r$ as shorthand for $\text{run}(t; a[1] \dots a[n]) = r$. The arguments are Nock nouns, not SKI terms: atoms in the identity and boolean laws, real formulas such as $+/\text{INC}$ in the arithmetic examples. A tap must not fire until its formula is a concrete noun.

Why quote, and why this order? Evaluating $[[A \ B]] = [2 \ [[B]] \ [[A]]]$ against any subject s gives $*[*[s \ [[B]]] \ *[s \ [[A]]]]$. Because $[[B]]$ and $[[A]]$ are closed, they ignore s and reduce to the values $\text{val}(B)$ and $\text{val}(A)$. The result is $*[\text{val}(B) \ \text{val}(A)]$ —apply value $\text{val}(A)$ to argument $\text{val}(B)$, i.e. A applied to B . The operands are reduced to values *before* application, which is what makes nested applications compose correctly; quoting a sub-application rather than evaluating it produces an “off-by-one error” (a residual K where the value was expected).

Correctness is demonstrated practically by induction on term structure. For application, the rule computes $\text{apply}(\text{val}(A), \text{val}(B))$, and by the induction hypothesis $\text{val}(A), \text{val}(B)$ are the values of A, B ; by §2 this is the value of $A \ B$. The translation is therefore a homomorphism from SKI application to Nock evaluation, under the call-by-value strategy forced by opcode 2’s eager semantics (see Section ??).

5 Worked Examples

All terms below are written in lambda form for legibility, elaborated to SKI by standard bracket abstraction (**Kiselyov2018**, **Kiselyov2018**), then compiled by the Nock SKI compiler and run. The results are the products actually computed.

A script which resolves the SKI terms to Nock formulas, runs them in a Nock 4K interpreter, and checks the results is available at [sigilante/skitrace](https://sigilante.github.io/skitrace).

5.1 Identity laws

From canonical SKI, we anticipate the following identity laws to hold true:

I 42	= 42	
S K K 42	= 42	:: S K K → I
S K S 42	= 42	:: S K * → I

The evaluation of S K K 42, worked in Nock SKI, is shown in Listings 1, 2, and 3. The process is mechanical and discursive. The evaluation of S K S 42 is similar and left as a proverbial exercise to the reader.

Listing 1: Compilation expansion of S K K 42.

```

★[42 [[S K K]]]
★[42 2 [[K]] [[S K]]]
                                [[A B]] = [2 [[B]] [[A]]]
★[42 2 [1 K] [[S K]]]
5                                [[K]] = [1 K]
★[42 2 [1 K] 2 [[K]] [[S]]]
                                [[A B]] = [2 [[B]] [[A]]]
★[42 2 [1 K] 2 [1 K] [[S]]]
                                [[K]] = [1 K]
10 ★[42 2 [1 K] 2 [1 K] 1 S]
                                [[S]] = [1 S]
★[42 2 [1 [1 1] 0 1] 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0
    1] [1 1] 0 1]
                                substitute S,K,I formulas

```

Listing 2: Evaluation of S K K 42.

```

*[[42 2 [1 [1 1] 0 1] 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0
  1] [1 1] 0 1]]
      Nock 2  *[[a 2 b c] = *[[a b] *[[a c]]
*[[*[[42 1 [1 1] 0 1] *[[42 2 [1 [1 1] 0 1] 1 [1 1 2] [1
  0 1] [1 1] 0 1]]]]
      Nock 1  *[[a 1 b] = b
5 *[[[[1 1] 0 1] *[[42 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1]
  [1 1] 0 1]]]]
      Nock 2  *[[a 2 b c] = *[[a b] *[[a c]]
*[[[[1 1] 0 1] *[[*[[42 1 [1 1] 0 1] *[[42 1 [1 1 2] [1 0
  1] [1 1] 0 1]]]]]]
      Nock 1  *[[a 1 b] = b
*[[[[1 1] 0 1] *[[[1 1] 0 1] *[[42 1 [1 1 2] [1 0 1] [1
  1] 0 1]]]]]]
10      Nock 1  *[[a 1 b] = b
*[[[[1 1] 0 1] *[[[1 1] 0 1] [1 1 2] [1 0 1] [1 1] 0
  1]]]]
      cons  *[[a [b c] d] = *[[a b c] *[[a d]]
*[[[[1 1] 0 1] *[[[1 1] 0 1] 1 1 2] *[[[1 1] 0 1] [1 0
  1] [1 1] 0 1]]]]
      Nock 1  *[[a 1 b] = b
15 *[[[[1 1] 0 1] [1 2] *[[[1 1] 0 1] [1 0 1] [1 1] 0 1]]]
      cons  *[[a [b c] d] = *[[a b c] *[[a d]]
*[[[[1 1] 0 1] [1 2] *[[[1 1] 0 1] 1 0 1] *[[[1 1] 0
  1] [1 1] 0 1]]]]
      Nock 1  *[[a 1 b] = b
*[[[[1 1] 0 1] [1 2] [0 1] *[[[1 1] 0 1] [1 1] 0 1]]]
20      cons  *[[a [b c] d] = *[[a b c] *[[a d]]
*[[[[1 1] 0 1] [1 2] [0 1] *[[[1 1] 0 1] 1 1] *[[[1 1]
  0 1] 0 1]]]]
      Nock 1  *[[a 1 b] = b
*[[[[1 1] 0 1] [1 2] [0 1] 1 *[[[1 1] 0 1] 0 1]]]
      Nock 0  *[[a 0 b] = /[b a]
25 *[[[[1 1] 0 1] [1 2] [0 1] 1 /[1 [[1 1] 0 1]]]]
      /[1 a] = a
*[[[[1 1] 0 1] [1 2] [0 1] 1 [1 1] 0 1]]
      cons  *[[a [b c] d] = *[[a b c] *[[a d]]
*[[[[[1 1] 0 1] 1 2] *[[[1 1] 0 1] [0 1] 1 [1 1] 0 1]]]
30      Nock 1  *[[a 1 b] = b
[2 *[[[1 1] 0 1] [0 1] 1 [1 1] 0 1]]]
      cons  *[[a [b c] d] = *[[a b c] *[[a d]]

```

```

[2 *[[[1 1] 0 1] 0 1] *[[[1 1] 0 1] 1 [1 1] 0 1]]
      Nock 0  *[[a 0 b] = /[b a]
35 [2 /[1 [[1 1] 0 1]] *[[[1 1] 0 1] 1 [1 1] 0 1]]
      /[1 a] = a
[2 [[1 1] 0 1] *[[[1 1] 0 1] 1 [1 1] 0 1]]
      Nock 1  *[[a 1 b] = b
[2 [[1 1] 0 1] [1 1] 0 1]
40 (= [2 K K], the identity combinator's value)

```

Listing 3: Application of S K K 42.

```

# applying val(S K K) to 42  ( *[42 val(S K K)] )
val(S K K) = [2 [[1 1] 0 1] [1 1] 0 1]

# evaluation of *[42 [2 [[1 1] 0 1] [1 1] 0 1]]
5  *[42 2 [[1 1] 0 1] [1 1] 0 1]
      Nock 2  *[[a 2 b c] = *[[a b] *[a c]]
      *[[42 [1 1] 0 1] *[42 [1 1] 0 1]]
      cons  *[[a [b c] d] = *[[a b c] *[a d]]
      *[[*[42 1 1] *[42 0 1]] *[42 [1 1] 0 1]]
10      Nock 1  *[[a 1 b] = b
      *[[1 *[42 0 1]] *[42 [1 1] 0 1]]
      Nock 0  *[[a 0 b] = /[b a]
      *[[1 /[1 42]] *[42 [1 1] 0 1]]
      /[1 a] = a
15  *[[1 42] *[42 [1 1] 0 1]]
      cons  *[[a [b c] d] = *[[a b c] *[a d]]
      *[[1 42] *[42 1 1] *[42 0 1]]
      Nock 1  *[[a 1 b] = b
      *[[1 42] 1 *[42 0 1]]
      Nock 0  *[[a 0 b] = /[b a]
20  *[[1 42] 1 /[1 42]]
      /[1 a] = a
      *[[1 42] 1 42]
      Nock 1  *[[a 1 b] = b
25  42

```

5.2 Church booleans

Conventional Nock utilizes atoms, but SKI combinators are pure construction; we therefore must utilize Church-style booleans and numerals within the particular three-opcode discipline we

follow here. Thus while this section is demonstrative of SKI in Nock, it evades the usual Nock idioms and in particular the economy afforded by the introduction of opcodes 3, 4, and 5.

A Church boolean is a two-argument function that selects one of its arguments. Because the combinators are pure construction, the truth values are instead represented as combinator sequences. With $\text{TRUE} = K$ and $\text{FALSE} = K\ I$:

```

TRUE p q
  = K p q
  = p

5 FALSE p q
   = K I p q
   = I q
   = q

10 TRUE 7 8 = 7
   FALSE 7 8 = 8

```

These are the canonical encodings from SKI praxis; here we probe with atom arguments because neither combinator applies its arguments as functions.

5.3 The B, C, W combinators

Elaborated from their defining lambda terms:

```

B = λ f g x. f (g x)    compose
C = λ f x y. f y x      transpose
W = λ f x. f x x        duplicate

```

Driving B with a real Nock increment $\text{inc} = [4\ 0\ 1]$ as payload, and C, W with atoms:

```

B inc inc 7 = 9      :: inc (inc 7)
C K 7 8     = 8      :: K 8 7 = 8
W K 7       = 7      :: K 7 7 = 7

```

These exercise three-variable abstraction and argument duplication. Note that the compiled SKI scaffolding remains pure o , 1, 2; the only 4 in B's run is the external increment supplied as data.²

²We here draw a sharp distinction between opcode 4 as $\text{inc} = [4\ 0\ 1]$

5.4 Church numerals and arithmetic

A Church numeral `church n` applies its first argument n times to its second. Numerals are built and incremented entirely within the fragment.³ Examples of Church numerals and their Nock SKI representations are given in Table 1. At this point, we will revert in part to the lambda calculus notation for clarity, but the underlying Nock SKI formulas are always available and are used in the actual evaluation. The successor function is defined by the usual combinator formula:

```
church n =  $\lambda f.x. f^n x$ 
succ =  $\lambda \lambda n.f.x. f (n f x)$ 
```

```
succ (church 0) = church 1
5 succ (church 1) = church 2
```

In explicit Nock SKI form, the Church numeral successor function becomes:

```
succ = [[1 1 2] [0 1] 1 [1 1] 0 1]
```

```
church 2 = [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0
1]□
```

```
church 02 = 2
```

```
5 succ (church 2) = [[1 2] [[1 2] [[1 2] [1 0 1] [1 1]
0 1] [1 1] 0 1] [1 1] 0 1]□
```

```
succ (church 2)□ = 3
```

Arithmetic over the Church numerals is defined by the usual combinator formulas:

```
:: in Church numerals
plus =  $\lambda \lambda \lambda m.n.f.x. m f (n f x)$ 
mult =  $\lambda \lambda \lambda m.n.f. m (n f)$ 
```

which operates on Nock atoms, and `succ` which operates on Church numerals. The two are distinct: `succ` maps a numeral-value to a numeral-value in 0,1,2, while `inc` maps an atom to an atom via opcode 4.

³A Church numeral is a value (formula), not an atom. To read it out as a Nock atom we decode it: apply it to the atom successor `inc = [4 0 1]` and the atom 0, writing `n = n inc 0`: `church 0 = 0` ; `SUCC*5 (church 0) = 5` ; `SUCC (church 2) = 3`. Decoding is the only step that uses opcode 4, and it is read-out, not arithmetic.

```

5  :: alternatively
   plus = λm n. m succ n
   mult = λm n. m (plus n) (church 0)

   :: in Nock decoding:
10 plus = [[1 1 1 2] [1 0 1] [1 1 1 1] [1 1] 0 1]
   mult = [[1 1 2] [1 0 1] [1 1 1] [1 1] 0 1]

   succ (church 2) = church 3

15 :: in Church numerals:
   church 0 succ 0 = 0
   church 3 succ 0 = [[1 2] [[1 2] [[1 2] 0 [1 1] 0 1]
                             [1 1] 0 1] [1 1] 0 1]
   church 5 succ 0 = 5
   (plus 2 3) succ 0 = 5
20 (mult 3 4) succ 0 = 12

   :: in Nock decoding:
   church 00 = 00
   church 03 = 30
25 church 05 = 50
   plus 2 03 = 50
   mult 3 04 = 12

```

Table 1: Church numerals and their Nock SKI representations. The numeral `church n` is a value (formula) that applies its first argument `n` times to its second. The decoding `n = n inc 0` produces the Nock atom corresponding to the numeral.

<code>n</code>	<code>□n□</code>	<code>church n</code>
0	0	<code>[1 0 1]</code>
1	1	<code>[[1 2] [1 0 1] [1 1] 0 1]</code>
2	2	<code>[[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1]</code>
3	3	<code>[[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]</code>
4	4	<code>[[1 2] [[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]</code>

The construction is Turing-complete, but it does not have laziness, normal-form readback, or sharing. Each of these properties marks a boundary where the fragment is genuinely insufficient and additional tooling is required.

6 Conclusion

This article makes concrete the claim that Nock opcodes 0, 1, and 2 are sufficient to represent the SKI combinator calculus and are thus Turing-complete (if insufficient to handle nouns natively). The embedding is a constructive proof that the opcode 0, 1, 2 fragment is Turing-complete: the deferred monograph the *Documentary History* expected is rooted in a handful of lines of Nock. The compiler is a homomorphism from SKI application to Nock evaluation, under a fixed call-by-value convention. The combinators are pure construction; opcode 2 appears only in the formula they assemble at application time, carried as quoted data. A second corollary locates the fragment's edge precisely: a conditional's *selection* is combinatory and lives in 0, 1, 2, but its *decision*—computing a boolean from data—does not, which motivates Nock's basis as an SKI core plus a minimal inspection and arithmetic kernel. The Nock ISA has been manifestly Turing-complete since its inception, but this exposition makes a new demonstration explicit and constructive.☒